Báo cáo tuần 7

1. Hàm cells_to_bboxes

Hàm cells_to_bboxes được sử dụng để chuyển đổi đầu ra của mô hình phát hiện đối tượng từ dạng biểu diễn theo ô lưới sang danh sách các bounding box ở hệ tọa độ toàn ảnh.

1.1. Bối cảnh

Trong các mô hình phát hiện đối tượng dạng YOLO, ảnh đầu vào được chia thành một lưới kích thước S nhân S. Mỗi ô lưới sẽ dự đoán một số bounding box tương ứng với các anchor. Các giá trị dự đoán bao gồm:

- Độ tin cậy có vật thể
- Tọa độ tương đối của bounding box trong ô lưới
- Xác suất các lớp

Tuy nhiên, các giá trị này chưa phải là tọa độ tuyệt đối trên ảnh, do đó cần chuyển đổi.

1.2. Đầu vào

- predictions: tensor chứa đầu ra của mô hình hoặc ground truth
- anchors: danh sách anchor box
- S: kích thước lưới
- is_preds: xác định dữ liệu là dự đoán hay ground truth
- output_format: định dạng đầu ra

```
def cells_to_bboxes(predictions, anchors, S, is_preds=True, output_format = 'xywh'):
    """
```

1.3. Xử lý với dữ liệu dự đoán

Khi is_preds bằng True, tensor predictions chứa các giá trị dạng logits.

Bước 1. Tách các thành phần

- objectness: xác suất tồn tại vật thể
- box_predictions: gồm x, y, w, h
- class_probs: xác suất các lớp

```
box_predictions = predictions[..., 1:5] # Lấy [x, y, w, h]
objectness = torch.sigmoid(predictions[..., 0:1]) # Lấy [obj]
class_probs = torch.sigmoid(predictions[..., 5:]) # Xác suất 20 classes
```

Bước 2. Chuẩn hóa xác suất

- Áp dụng sigmoid cho objectness và class_probs để đưa về khoảng từ 0 đến 1
- Xác định lớp dự đoán bằng cách chọn lớp có xác suất lớn nhất

Bước 3. Tính score

Score của bounding box được tính bằng:

score bằng objectness nhân với xác suất lớp cao nhất

```
best_class_prob, best_class = torch.max(class_probs, dim=-1, keepdim=True)
final_scores = objectness * best_class_prob
```

Bước 4. Biến đổi tọa độ

- x và y được đưa qua sigmoid để nằm trong phạm vi một ô lưới
- w và h được biến đổi bằng hàm mũ và nhân với anchor tương ứng

```
# Biến đổi thô (logits) thành tọa độ
anchors = anchors.reshape(1, num_anchors, 1, 1, 2)
box_predictions[..., 0:2] = torch.sigmoid(box_predictions[..., 0:2])
box_predictions[..., 2:4] = torch.exp(box_predictions[..., 2:4]) * anchors
```

Mục đích của bước này là đưa các giá trị từ không gian logits về kích thước thực tế của bounding box.

1.4. Xử lý với ground truth

Khi is_preds bằng False:

- Không cần áp dụng sigmoid hoặc exp
- Các giá trị đã được chuẩn hóa sẵn
- Trực tiếp sử dụng các giá trị x, y, w, h và class

```
# Tensor target có cấu trúc: [obj, x_offset, y_offset, w_grid, h_grid, class_index]
box_predictions = predictions[..., 1:5]
final_scores = predictions[..., 0:1] # Điểm tự tin của GT luôn là 1 (hoặc 0)
best_class = predictions[..., 5:6]   # Chứa sẵn ID của class (ví dụ: 14 là Person)
```

1.5. Chuyển sang hệ tọa độ toàn ảnh

Hàm tạo một tensor grid biểu diễn vị trí của từng ô trong lưới.

Tọa độ được tính như sau:

- x bằng x_offset cộng vị trí ô theo chiều ngang, sau đó chia cho S
- y tương tự theo chiều dọc
- w và h chia cho S

```
grid = torch.arange(S).repeat(B, num_anchors, S, 1).unsqueeze(-1).to(predictions.device)

# Đưa x, y, w, h về (0.0 đến 1.0)
x = (box_predictions[..., 0:1] + grid) / S
y = (box_predictions[..., 1:2] + grid.permute(0, 1, 3, 2, 4)) / S
w_h = box_predictions[..., 2:4] / S;
```

Sau bước này, toàn bộ tọa độ được chuẩn hóa về khoảng từ 0 đến 1 theo kích thước ảnh.

1.6. Chuyển đổi định dạng

Tùy theo output_format:

- Nếu là xywh:
    - trả về tâm và kích thước bounding box
- Nếu là xyxy:
    - chuyển sang tọa độ góc trái trên và góc phải dưới
    - áp dụng giới hạn để đảm bảo nằm trong ảnh

```
if output_format == 'xywh':
    converted_bboxes = torch.cat((best_class, final_scores, x, y, w_h), dim=-1).reshape(B, num_anchors * S * S, 6)
else:
    x1 = x - w_h[..., 0:1] / 2
    y1 = y - w_h[..., 1:2] / 2
    x2 = x + w_h[..., 0:1] / 2
    y2 = y + w_h[..., 1:2] / 2
    # clamp
    x1 = x1.clamp_(0, 1)
    y1 = y1.clamp_(0, 1)
    x2 = x2.clamp_(0, 1)
    y2 = y2.clamp_(0, 1)
    converted_bboxes = torch.cat((best_class, final_scores, x1, y1, x2, y2), dim=-1).reshape(B, num_anchors * S * S, 6)
```

1.7. Kết quả

Hàm trả về danh sách bounding box với định dạng:

class, score, tọa độ bounding box

2. Hàm get_evaluation_bboxes

Hàm này được sử dụng để thu thập toàn bộ bounding box dự đoán và ground truth từ dataset nhằm phục vụ việc đánh giá mô hình.

2.1. Mục đích

- Chạy mô hình trên toàn bộ dữ liệu
- Chuyển đổi output thành bounding box
- Lọc các bounding box không cần thiết
- Áp dụng Non Maximum Suppression
- Chuẩn bị dữ liệu để tính các metric đánh giá

2.2. Đầu vào

- loader: dataloader chứa dữ liệu
- model: mô hình đã huấn luyện
- iou_threshold: ngưỡng dùng trong NMS
- anchors: anchor box
- threshold: ngưỡng confidence để lọc box

```
def get_evaluation_bboxes(
    loader,
    model,
    iou_threshold,
    anchors,
    threshold,
    device="cuda",
):
```

2.3. Quy trình xử lý

Bước 1. Chuyển model sang chế độ đánh giá

- Tắt gradient
- Đảm bảo kết quả ổn định khi inference

Bước 2. Duyệt qua từng batch

- Lấy dữ liệu ảnh và nhãn
- Thực hiện forward pass để lấy predictions

```
for batch_idx, (x, labels) in enumerate(tqdm(loader)):
    x = x.to(device)

    with torch.no_grad():
        predictions = model(x)
```

Bước 3. Xử lý nhiều scale

Mô hình YOLO dự đoán trên nhiều scale khác nhau.

Với mỗi scale:

- Chuyển predictions thành bounding box bằng hàm cells_to_bboxes
- Chuyển ground truth tương tự

Bước 4. Gom bounding box theo từng ảnh

- Mỗi ảnh có thể có nhiều bounding box từ nhiều scale
- Tất cả được gom lại thành một danh sách

```
    anchor = torch.tensor([*anchors[i]]).to(device) * S
    boxes_scale_i = cells_to_bboxes(
        predictions[i], anchor, S=S, is_preds=True, output_format='xyxy'
    )
    for idx, (box) in enumerate(boxes_scale_i):
        bboxes[idx] += box


    true_boxes = cells_to_bboxes(
        labels[i], anchor, S=S, is_preds=False, output_format='xyxy' # here
    )
    for idx, box, in enumerate(true_boxes):
        true_bboxes[idx] += box
```

Bước 5. Lọc theo class và confidence

Với mỗi class:

- Chỉ giữ các box có score lớn hơn threshold
- Chọn tối đa 400 box có score cao nhất để giảm tỉ lệ có lớp nào có số box áp đảo các lớp còn lại

Mục đích là giảm số lượng box trước khi áp dụng NMS

```
# pre-nms filtering: 400 box per class
for label in range(config.NUM_CLASSES):
    label_mask = class_preds == label
    scores_label = scores[label_mask] # (N_label,)
    coords_label = coords[label_mask] # (N_label, 4)
    class_preds_label = class_preds[label_mask] # (N_label,)
    assert scores_label.shape[0] == coords_label.shape[0] == class_preds_label.shape[0]

    keep_idxs = scores_label > threshold

    scores_label = scores_label[keep_idxs]
    coords_label = coords_label[keep_idxs]
    class_preds_label = class_preds_label[keep_idxs]
    if coords_label.shape[0] == 0:
        continue

    score, top_k_idxs = scores_label.topk(min(400, scores_label.shape[0]))
    coords_label = coords_label[top_k_idxs]
    class_preds_label = class_preds_label[top_k_idxs]
    pred_coords.append(coords_label)
    pred_scores.append(score)
    pred_labels.append(class_preds_label)

if len(pred_coords) == 0: continue
pred_coords = torch.cat(pred_coords, dim=0)
pred_scores = torch.cat(pred_scores, dim=0)
pred_labels = torch.cat(pred_labels, dim=0)
```

.

Bước 6. Áp dụng Non Maximum Suppression

Sử dụng batched NMS:

- Loại bỏ các box trùng nhau dựa trên IoU
- Thực hiện riêng cho từng class
- Giữ lại các box có score cao nhất
- Sau đó chỉ giữ tối đa 100 box tốt nhất vì dự đoán có thể sinh ra rất nhiều box nên việc loại bỏ là cần thiết

```python
TOK_K = config.BOXES_PER_IMAGE
keep = post_nms_keep_indices[:TOK_K] # keep top 100 after nms

pred_coords = pred_coords[keep]
pred_scores = pred_scores[keep]
pred_labels = pred_labels[keep]

selected = torch.cat(S
    [
        pred_labels.unsqueeze(1),
        pred_scores.unsqueeze(1),
        pred_coords,
    ],
    dim=1,
)
```

Bước 7. Lưu kết quả

- Bounding box dự đoán được lưu kèm chỉ số ảnh
- Ground truth cũng được lưu tương tự

Định dạng lưu trữ:

image_index, class, score, tọa độ

2.4. Kết quả

Hàm trả về hai danh sách:

- all_pred_boxes: toàn bộ bounding box dự đoán
- all_true_boxes: toàn bộ ground truth

Các dữ liệu này được sử dụng để tính các metric như mean Average Precision

3. Hàm compute_metrics

Hàm compute_metrics được sử dụng để đánh giá hiệu năng của mô hình phát hiện đối tượng thông qua các chỉ số định lượng quan trọng, bao gồm mean Average Precision, Precision, Recall và F1-score.

3.1. Mục đích

Hàm này thực hiện việc:

- Tính toán mean Average Precision trên từng lớp
- Tính Precision, Recall và F1-score tại một ngưỡng confidence xác định
- Tổng hợp các chỉ số đánh giá theo từng lớp và toàn bộ dataset

Nhờ đó, hàm cung cấp cái nhìn toàn diện về chất lượng của mô hình.

3.2. Đầu vào

- pred_boxes: danh sách hoặc tensor chứa các bounding box dự đoán
- true_boxes: danh sách hoặc tensor chứa các bounding box ground truth
- iou_threshold: ngưỡng IoU để xác định True Positive
- score_threshold: ngưỡng confidence để tính Precision và Recall
- box_format: định dạng tọa độ bounding box
- num_classes: số lượng lớp

Mỗi bounding box có cấu trúc:

image_index, class, score, tọa độ

```
def compute_metrics(
    pred_boxes,
    true_boxes,
    iou_threshold=0.5,
    score_threshold=0.5,
    box_format="xyxy",
    num_classes=20,
):
```

3.3. Tiền xử lý

- Chuyển đổi dữ liệu từ list sang tensor nếu cần thiết
- Khởi tạo các biến:
    - average_precisions để lưu Average Precision của từng lớp

- per_class_metrics để lưu các chỉ số theo lớp
- total_TP_all, total_FP_all, total_FN_all để tính toán tổng thể

```
average_precisions = []
per_class_metrics = {}

total_TP_all = 0
total_FP_all = 0
total_FN_all = 0

epsilon = 1e-6

# Chuyển đổi sang tensor nếu input đang là list để tăng tốc
if isinstance(pred_boxes, list):
    pred_boxes = torch.tensor(pred_boxes)
if isinstance(true_boxes, list):
    true_boxes = torch.tensor(true_boxes)
```

3.4. Quy trình xử lý theo từng lớp

Hàm lặp qua từng lớp để tính toán độc lập.

3.4.1. Lọc dữ liệu theo lớp

- Lấy các bounding box dự đoán thuộc lớp đang xét
- Lấy các ground truth tương ứng

Nếu không tồn tại ground truth, lớp đó sẽ bị bỏ qua.

```
for c in range(num_classes):
    detections = pred_boxes[pred_boxes[:, 1] == c]
    ground_truths = true_boxes[true_boxes[:, 1] == c]

    total_true_bboxes = ground_truths.shape[0]
    if total_true_bboxes == 0:
        continue
```

3.4.2. Khởi tạo bộ đếm ground truth

- Sử dụng cấu trúc dữ liệu để theo dõi mỗi ground truth đã được khớp hay chưa

- Mỗi ground truth chỉ được phép ghép với một prediction duy nhất

Điều này giúp tránh việc một đối tượng bị đếm nhiều lần.

```
amount_bboxes = Counter(ground_truths[:, 0].tolist())
for key, val in amount_bboxes.items():
    amount_bboxes[key] = torch.zeros(val)
```

3.4.3. Sắp xếp dự đoán

- Các prediction được sắp xếp theo confidence giảm dần
- Đây là cơ sở để xây dựng đường cong Precision Recall

```
detections = detections[detections[:, 2].argsort(descending=True)]
```

3.4.4. Xác định True Positive và False Positive

Với mỗi prediction:

- Lấy tất cả ground truth thuộc cùng ảnh
- Tính IoU giữa prediction và từng ground truth
- Chọn ground truth có IoU lớn nhất

Sau đó:

- Nếu IoU lớn hơn ngưỡng:
    - Nếu ground truth chưa được khớp:
        - đánh dấu là True Positive
    - Nếu đã được khớp:
        - đánh dấu là False Positive
- Nếu IoU nhỏ hơn ngưỡng:
    - đánh dấu là False Positive

```python
for detection_idx, detection in enumerate(detections):
    ground_truth_img = ground_truths[ground_truths[:, 0] == detection[0]]

    best_iou = 0
    best_gt_idx = -1
    img_idx = detection[0].item()

    for idx, gt in enumerate(ground_truth_img):
        iou = intersection_over_union(
            detection[3:], gt[3:], box_format=box_format
        )
        if iou > best_iou:
            best_iou = iou
            best_gt_idx = idx

    if best_iou > iou_threshold:
        if amount_bboxes[img_idx][best_gt_idx] == 0:
            TP[detection_idx] = 1
            amount_bboxes[img_idx][best_gt_idx] = 1
        else:
            FP[detection_idx] = 1
    else:
        FP[detection_idx] = 1
```

3.5. Tính mean Average Precision

3.5.1. Tích lũy TP và FP

- Tính tổng tích lũy TP và FP theo thứ tự confidence
- Mỗi bước tương ứng với một ngưỡng confidence khác nhau

3.5.2. Tính Precision và Recall theo đường cong

- Recall được tính bằng TP chia tổng số ground truth
- Precision được tính bằng TP chia tổng TP và FP

3.5.3. Chuẩn hóa đường cong

- Thêm điểm đầu với recall bằng 0 và precision bằng 1
- Giúp đường cong bắt đầu từ gốc tọa độ

3.5.4. Tính Average Precision

- Sử dụng tích phân số để tính diện tích dưới đường cong Precision Recall
- Giá trị này chính là Average Precision của lớp

```
TP_cumsum = torch.cumsum(TP, dim=0)
FP_cumsum = torch.cumsum(FP, dim=0)

recalls_curve = TP_cumsum / (total_true_bboxes + epsilon)
precisions_curve = TP_cumsum / (TP_cumsum + FP_cumsum + epsilon)

# Thêm điểm đầu cho đồ thị
p_c = torch.cat((torch.tensor([1.0]), precisions_curve))
r_c = torch.cat((torch.tensor([0.0]), recalls_curve))

AP = torch.trapz(p_c, r_c)
average_precisions.append(AP)
```

3.6. Tính Precision, Recall và F1-score tại ngưỡng score_threshold

Khác với mAP, các chỉ số này được tính tại một ngưỡng confidence cố định.

3.6.1. Lọc prediction theo ngưỡng

- Chỉ giữ các prediction có score lớn hơn score_threshold

3.6.2. Tính TP và FP tại ngưỡng

- Đếm số lượng True Positive và False Positive trong tập đã lọc

3.6.3. Tính các chỉ số

- Precision bằng TP chia tổng TP và FP
- Recall bằng TP chia tổng ground truth
- False Negative bằng tổng ground truth trừ TP
- F1-score là trung bình điều hòa giữa Precision và Recall

3.6.4. Lưu kết quả theo lớp

- Lưu lại các giá trị Precision, Recall và F1-score cho từng lớp

```
# Tính Precision/Recall tại score_threshold
# Tìm những index mà tại đó confidence > score_threshold
keep_indices = detections[:, 2] > score_threshold

# Tổng số TP và FP tại ngưỡng score_threshold này
# Chúng ta dùng mask keep_indices để lọc mảng TP, FP ban đầu
tp_at_thresh = TP[keep_indices].sum().item()
fp_at_thresh = FP[keep_indices].sum().item()

# Tính P, R tại ngưỡng này
p_class = tp_at_thresh / (tp_at_thresh + fp_at_thresh + epsilon)
r_class = tp_at_thresh / (total_true_bboxes + epsilon)
fn_class = total_true_bboxes - tp_at_thresh
f1_class = 2 * p_class * r_class / (p_class + r_class + epsilon)

per_class_metrics[c] = {
    "precision": p_class,
    "recall": r_class,
    "f1": f1_class,
}
```

3.7. Tính toán tổng thể

Các chỉ số tổng thể được tính theo phương pháp micro average.

3.7.1. Cộng dồn

- Cộng tất cả TP, FP và FN từ các lớp

```
# Cộng dồn để tính Micro-average (cũng tại ngưỡng score_threshold)
total_TP_all += tp_at_thresh
total_FP_all += fp_at_thresh
total_FN_all += fn_class
```

3.7.2. Tính các chỉ số

- Precision tổng thể
- Recall tổng thể
- F1-score tổng thể

Các chỉ số này phản ánh hiệu năng chung của mô hình trên toàn bộ dataset.

```
# Overall Metrics
overall_precision = total_TP_all / (total_TP_all + total_FP_all + epsilon)
overall_recall = total_TP_all / (total_TP_all + total_FN_all + epsilon)
overall_f1 = 2 * overall_precision * overall_recall / (
    overall_precision + overall_recall + epsilon
)
```

3.8. Kết quả đầu ra

Hàm trả về một dictionary gồm:

- map: giá trị mean Average Precision trung bình trên các lớp
- per_class_metric: các chỉ số Precision, Recall, F1 theo từng lớp
- overall: các chỉ số Precision, Recall, F1 tổng thể

```
return {
    "map": (sum(average_precisions) / len(average_precisions)).item() if average_precisions else 0.0,
    "per_class_metric": per_class_metrics,
    "overall": {
        "precision": overall_precision,
        "recall": overall_recall,
        "f1": overall_f1,
    },
}
```

4. Kết quả thực nghiệm và nhận xét

```
[ SUMMARY METRICS ]
mAP: 0.6139
-------------------------------------------------
Class       | Precision   | Recall      | F1
-------------------------------------------------
aeroplane   | 0.2051      | 0.7942      | 0.3261
bicycle     | 0.4926      | 0.7686      | 0.6004
bird        | 0.2670      | 0.7847      | 0.3984
boat        | 0.0709      | 0.7354      | 0.1293
bottle      | 0.0436      | 0.7199      | 0.0822
bus         | 0.6054      | 0.7008      | 0.6496
car         | 0.0835      | 0.8871      | 0.1526
cat         | 0.6744      | 0.8622      | 0.7568
chair       | 0.0936      | 0.7227      | 0.1657
cow         | 0.4943      | 0.6535      | 0.5628
diningtable | 0.3328      | 0.6656      | 0.4437
dog         | 0.5940      | 0.8585      | 0.7022
horse       | 0.5564      | 0.7494      | 0.6386
motorbike   | 0.5693      | 0.7127      | 0.6330
person      | 0.0948      | 0.9110      | 0.1717
pottedplant | 0.0877      | 0.7500      | 0.1571
sheep       | 0.2152      | 0.7846      | 0.3377
sofa        | 0.4662      | 0.6439      | 0.5408
train       | 0.4912      | 0.8311      | 0.6175
tvmonitor   | 0.3119      | 0.7673      | 0.4436
-------------------------------------------------
OVERALL     | 0.1144      | 0.8198      | 0.2007
-------------------------------------------------
```

4.1. Kết quả tổng thể

Kết quả đánh giá mô hình cho thấy giá trị mean Average Precision đạt 0.6139. Đây là mức hiệu năng tương đối tốt đối với bài toán phát hiện đối tượng với nhiều lớp và dữ liệu phức tạp.

Ở mức tổng thể, các chỉ số đạt được như sau:

- Precision đạt 0.1144
- Recall đạt 0.8198
- F1 score đạt 0.2007

Kết quả này cho thấy mô hình có khả năng phát hiện được phần lớn đối tượng trong ảnh, thể hiện qua Recall cao. Tuy nhiên Precision thấp cho thấy số lượng dự đoán sai còn nhiều.

4.2. Phân tích theo từng lớp

Kết quả chi tiết cho từng lớp cho thấy sự khác biệt rõ rệt về hiệu năng giữa các đối tượng.

Nhóm lớp có hiệu năng tốt

Một số lớp đạt kết quả tốt cả về Precision và Recall, dẫn đến F1 score cao:

- cat với F1 đạt 0.7568
- dog với F1 đạt 0.7022
- bus với F1 đạt 0.6496
- horse với F1 đạt 0.6386
- motorbike với F1 đạt 0.6330

Các lớp này thường có đặc điểm hình dạng rõ ràng, dễ phân biệt và xuất hiện nhiều trong dữ liệu huấn luyện.

Nhóm lớp có Recall cao nhưng Precision thấp

Một số lớp có Recall rất cao nhưng Precision thấp, ví dụ:

- person với Recall đạt 0.9110 nhưng Precision chỉ 0.0948
- car với Recall đạt 0.8871 nhưng Precision chỉ 0.0835
- aeroplane với Recall đạt 0.7942 nhưng Precision chỉ 0.2051

Điều này cho thấy mô hình phát hiện được hầu hết các đối tượng thật, nhưng đồng thời cũng tạo ra nhiều dự đoán sai. Nguyên nhân có thể do:

- nhiều bounding box trùng lặp
- nhầm lẫn với các đối tượng có hình dạng tương tự
- ngưỡng confidence chưa được tối ưu

Nhóm lớp có hiệu năng thấp

Một số lớp có cả Precision và F1 score thấp:

- bottle với F1 đạt 0.0822
- boat với F1 đạt 0.1293
- chair với F1 đạt 0.1657
- pottedplant với F1 đạt 0.1571

Các lớp này thường có đặc điểm:

- kích thước nhỏ hoặc không rõ ràng
- hình dạng đa dạng
- dễ bị che khuất hoặc lẫn với nền

4.3. Nhận xét chung

Từ các kết quả trên có thể rút ra một số nhận xét quan trọng:

Thứ nhất, mô hình có xu hướng ưu tiên Recall hơn Precision. Điều này có nghĩa là mô hình cố gắng phát hiện càng nhiều đối tượng càng tốt, nhưng đánh đổi bằng việc tăng số lượng dự đoán sai.

Thứ hai, sự chênh lệch giữa các lớp cho thấy mô hình chưa học tốt các đặc trưng của một số đối tượng khó. Các lớp có kích thước nhỏ hoặc hình dạng không ổn định thường có hiệu năng thấp hơn.

Thứ ba, giá trị mean Average Precision ở mức trung bình khá cho thấy mô hình có khả năng tổng quát hóa tốt, tuy nhiên vẫn còn chỗ để cải thiện.